

核心工作流

使用 Kimi API 的 Partial Mode

📄 复制页面



有些时候，我们希望 Kimi 大模型能顺着给定的语句继续往下说，例如，在某些客服场景，我们希望智能机器人客服每一句的开头都是“尊敬的用户您好”，对于这样的需求，Kimi API 提供了 Partial Mode。我们用具体的代码来讲解 Partial Mode 是如何运作的：

python **node.js**

KIMI 开放平台



```
api_key = "MOONSHOT_API_KEY", # 在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台
base_url = "https://api.moonshot.cn/v1",
)

completion = client.chat.completions.create(
    model = "kimi-k2.6",
    messages = [
        {"role": "system", "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你"},
        {"role": "user", "content": "你好? "},
        {
            "partial": True, # <-- 通过 partial 参数，开启 Partial Mode
            "role": "assistant", # <-- 我们在用户提问之后添加一条 role=assistant 的消息
            "content": "尊敬的用户您好，", # <-- 通过 content 把话"喂到 Kimi 大模型嘴里"
        },
    ],
)

# Kimi 大模型会顺着"喂到嘴里的话"继续说下去，因此我们需要手动将喂给 Kimi 大模型的话拼接到最后生
print("尊敬的用户您好，" + completion.choices[0].message.content)
```

我们总结一下使用 Partial Mode 的要点：

1. 在 messages 列表尾部添加一条额外的 message，设置 `role=assistant`、`partial=True`；
2. 将需要喂给 Kimi 大模型的内容放置在 `content` 字段中，Kimi 大模型会强制以 `content` 的内容开头开始生成回复；
3. 将步骤 2 中的 `content` 拼接 to Kimi 大模型生成的内容之前，组成完整的回复；

在调用 Kimi API 的过程中，可能会出现由于对输入和输出 Tokens 数量的预估出现偏差，导致 `max_tokens` 字段的值被设置过低，导致 Kimi 大模型不能完整地输出回复内容（这种情况下，`finish_reason` 的值为 `length`，即 Kimi 大模型生成的回复所占用的 Tokens 数量大于请求设置的 `max_tokens` 值）；此时，如果你对已经输出的内容感到满意，想让 Kimi 大模型顺着已经输出的内容继续输出剩余内容，那么 Partial Mode 就可以派上用场。

```
>
from openai import OpenAI

client = OpenAI(
    api_key = "MOONSHOT_API_KEY", # 在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台
    base_url = "https://api.moonshot.cn/v1",
)

completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=[
        {"role": "user", "content": "请背诵完整的出师表。"},
    ],
    max_tokens=1200, # <-- 注意这里，我们设置一个较小的 max_tokens 的值，以观察 Kimi 大
)

if completion.choices[0].finish_reason == "length": # <-- 当内容被截断时，finish_re
    prefix = completion.choices[0].message.content
    reasoning_content = completion.choices[0].message.reasoning_content
    print(prefix, end="") # <-- 在这里，你将看到被截断的部分输出内容
    print("「继续输出----->」 ")
    completion = client.chat.completions.create(
        model="kimi-k2.6",
        messages=[
            {"role": "user", "content": "请背诵完整的出师表。"},
            {"role": "assistant", "content": prefix, "partial": True, "reasoning_c
        ],
        max_tokens=86400, # <-- 注意这里，我们将 max_tokens 的值设置为一个较大的值，以确
    )
    print(completion.choices[0].message.content) # <-- 在这里，你将看到 Kimi 大模型顺
```

`name` 指定的角色的口吻输出内容。我们用一个使用 Kimi 大模型进行角色扮演的例子来说明

KIMI 开放平台的 `name` 字段应该如何使用，在这个例子中，我们使用明日方舟里的凯尔希医生为例。我们通过设置 `"name": "凯尔希"` 来更好地保持角色的一致性，这里的 `name` 字段是输出内容前缀的一部分，它会让 Kimi 大模型以凯尔希作为自己的角色进行输出：

python node.js

```
from openai import OpenAI

client = OpenAI(
    api_key="$MOONSHOT_API_KEY",
    base_url="https://api.moonshot.cn/v1",
)

completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=[
        {
            "role": "system",
            "content": "下面你扮演凯尔希，请用凯尔希的语气和我对话。凯尔希是手机游戏《明日方舟》中的角色。",
        },
        {
            "role": "user",
            "content": "你怎么看待特蕾西娅和阿米娅？",
        },
        {
            "partial": True,
            "role": "assistant",
            "name": "凯尔希",
            "content": "",
        },
    ],
    max_tokens=65536,
)

print(completion.choices[0].message.content)
```

还有一些帮助大模型在长时间对话中保持角色扮演一致性的通用方法：

KIMI 开放平台



- 提供清晰的角色描述，例如上面我们所做的那样，在设置角色时，详细介绍他们的个性、背景以及可能具有的任何具体特征或怪癖，这将有助于 Kimi 大模型更好地理解 and 模仿角色；
- 增加关于其要扮演的角色的细节，例如说话的语气、风格、个性，甚至背景，如背景故事和动机。例如上面我们提供了一些凯尔希的语录；
- 指导在各种情况下如何行动：如果预计角色会遇到某些特定类型的用户输入，或者希望控制模型在角色扮演互动中的某些情况下的输出，则应在系统提示词 system prompt 中提供明确的指令和指南，说明该角色在这些情况下应如何行动；
- 如果对话的轮次非常长，你还可以定期使用系统提示词 system prompt 强化角色的设定，特别是当模型开始产生一些偏离时，例如：

`python` `node.js`

KIMI 开放平台



```
import openai

api_key="$MOONSHOT_API_KEY",
base_url="https://api.moonshot.cn/v1",
)

completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=[
        {
            "role": "system",
            "content": "下面你扮演凯尔希，请用凯尔希的语气和我对话。凯尔希是手机游戏《明日方舟》中的角色。",
        },
        {
            "role": "user",
            "content": "你怎么看待特蕾西娅和阿米娅？",
        },
    ],

    # 假设这中间产生了非常多轮的对话
    # ...

    {
        "role": "system",
        "content": "下面你扮演凯尔希，请用凯尔希的语气和我对话。凯尔希是手机游戏《明日方舟》中的角色。",
    },
    {
        "partial": True,
        "role": "assistant",
        "name": "凯尔希",
        "content": "",
    },
],
    max_tokens=65536,
)

print(completion.choices[0].message.content)
```

 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

KIMI 开放平台



< JSON Mode

自动断线重连 >

>

